

From traditional SDLC to AI-native

What has to change, in what
order, and what it costs to skip
a step.

Three constraints, not seven

The list you gave us collapses into three families. They need different answers, and only one of them is a tooling problem.

Belief

Nobody trusts the output, and everybody expects miracles. Those are the same mistake made in opposite directions.

Capability

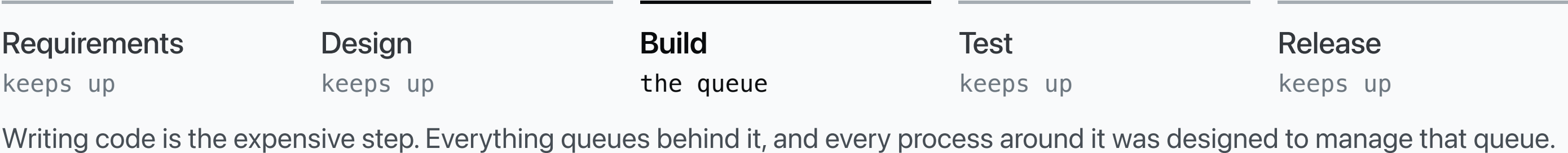
No training, no shared way of working, and processes built for a world where writing code was the expensive part.

Foundation

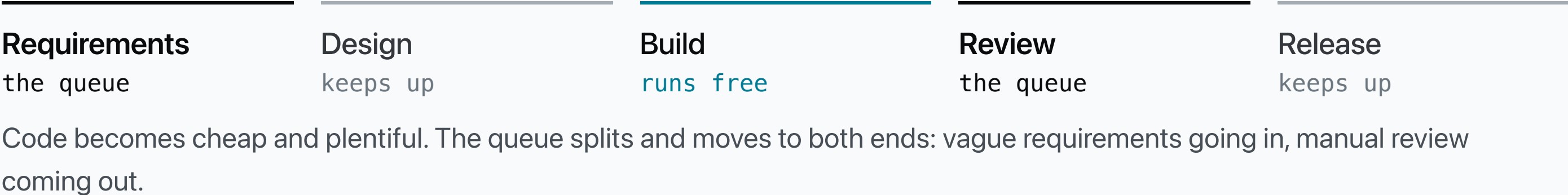
A codebase that resists change, and a Confluence nobody reads. An agent inherits both on its first prompt.

The bottleneck moves. It does not disappear.

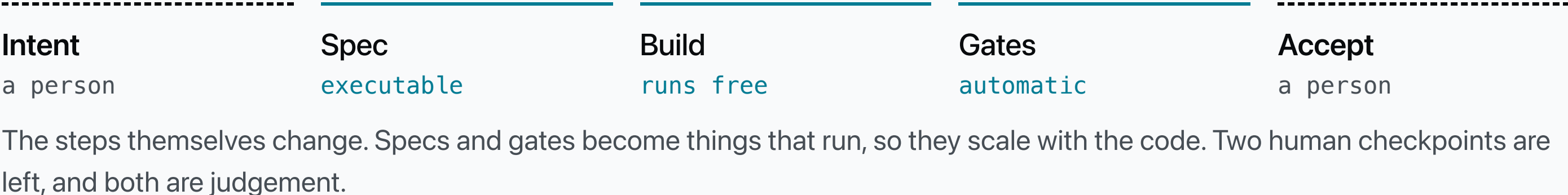
01
Traditional



02
Agentic



03
AI-native



Isabel is at 01. Buying seats without the rest lands you at 02.

When code stops being the constraint, everything you never automated becomes the constraint.

Agents do not take work out of the pipeline. They move it
upstream into requirements and downstream into verification.
At Isabel both of those are manual today.

Which is why tooling on its own changes nothing.

Four things, in this order

01

Quality gates

Make quality visible and automatic in CI/CD. A minimum requirement, not a later phase.

02

Real access

Replace the thirty dollar cap with a seat that can actually run agentic work.

03

Platform

One supported install and one codified way of working, shipped to every team.

04

Training

Teach the teams to work with agents rather than around them.

The order matters. Access without gates buys you more code and less confidence.

Quality has to be visible before it can be delegated

An agent produces more code in a day than a team used to produce in a week. Human review does not scale with that. Gates do.

Make it visible

SonarQube or SonarCloud on every repository, with the result on the pull request rather than in a report nobody opens.

Make it blocking

Coverage, duplication and security findings fail the build. A gate that only warns is a gate that gets ignored.

Make it the entry ticket

No team starts agentic work on a repository that does not have it. This is the one thing we would ask you to mandate centrally.

Thirty dollars is not a budget, it is a ceiling

An engineer waiting on a rate limit is the most expensive line item in this deck. The cap costs more than it saves.

Izzy at thirty dollars

What a developer has today

Enough for a handful of questions a day

Not enough for an agent that works for an hour

Teams ration themselves and stop trying

A working seat

What the work actually needs

A Claude subscription at around a hundred dollars

Local models for routine and sensitive work

A different order of capacity, for a rounding error on a salary

The platform is the offering

One supported way to install and run Claude Code, with your way of working built into it.

One install

One managed setup per developer, not twelve teams guessing.

A codified way of working

Your conventions as configuration, not as a Confluence page.

Skills and agents

What your teams repeat, packaged once. New joiners inherit it.

Tooling in reach

Your stack reachable through Claude plugins and MCP.

A tool changes what is possible. Training changes what is done.

Most of the disappointment with AI inside Isabel is not the model. It is people using an agent like a search box and concluding it is not very good. That is fixable in days, and it is the cheapest item on this list.

Existing staff first. New joiners will arrive with the habits already formed.

The last constraint is not technical

We can install a platform. We cannot install a willingness to change. This is the item that decides whether the other four land.

“We have always done it this way”

Answered by input from outside the team. Someone who has already made the move, in the room, not a memo about it.

“I do not trust it, my code is better”

Answered by demonstration. A real repository, a real change, measured against what the team would have shipped. Evidence beats argument.

“I am not going to”

Sometimes neither of those lands. Then it stops being a change programme and becomes a staffing decision, and that one is yours.

How we would start

01

Baseline

Gates on one repository per team, and an honest read of what that codebase does to an agent.

02

Seats

In parallel. Real access for one team, so the argument stops being theoretical.

03

Platform

The install, the way of working, the first skills and agents. Built around what the pilot team learned.

04

Roll out

Team by team, with training in front of each one rather than after it.

The sequence matters more than the dates. We would set those with you.

Which team goes first?

One team, one repository, gates and real seats. That is enough to settle this with evidence instead of opinion.

hello@smartagents.be

smartagents.be

Thank you